ÉCOLE NATIONALE DE COMMERCE ET DE GESTION DE SETTAT

Université Hassan Ier — Settat, Maroc

Compte Rendu de Projet

Détection de Fraude dans les Notes de Frais

Comparaison de trois algorithmes de Machine Learning

Régression Logistique • Random Forest • Isolation Forest

Module	Intelligence Artificielle
Filière	CAC 2 — Semestre 8
Réalisé par	Hajbi Douae | Fatima Ezzahraa Mache
Encadrant	M. Abderrahim Larhlimi
Période	Semestre 8 — Année universitaire 2025–2026

ENCG Settat — Intelligence Artificielle | Année Universitaire 2025–2026

Table des Matières

1. Introduction et Contexte 3
1.1. Présentation de l'étude 3
1.2. Présentation du jeu de données 3
2. Algorithme 1 — Régression Logistique 4
3. Algorithme 2 — Random Forest 5
4. Algorithme 3 — Isolation Forest 7
5. Comparaison et Évaluation Globale 9
6. Synthèse et Recommandation 10
A. Code Source — fraud_analysis.py 11

1. Introduction et Contexte

1.1. Présentation de l'étude

Ce rapport présente une analyse comparative approfondie des performances de trois algorithmes de Machine Learning appliqués à la détection de fraude dans les notes de frais et de remboursements. Les données utilisées constituent un jeu hybride de 5 000 transactions simulant des caractéristiques réalistes, inspirées du Expense Fraud Detection Dataset de Kaggle, enrichies de métadonnées RH (seuils d'alerte, distance, historique d'anomalies).

Les trois algorithmes évalués sont :

- **Régression Logistique — Modèle baseline linéaire supervisé**
- Random Forest — Ensemble learning supervisé
- Isolation Forest — Détection d'anomalies non supervisée

1.2. Présentation du jeu de données

Le jeu de données est généré via le script fraud_analysis.py et comprend les variables suivantes :

Tab. 1 — Variables du jeu de données simulé

Variable	Distribution	Description
montant	Log-normale	Montant de la dépense (€)
depense_weekend	Binomiale	Dépense effectuée le week-end
distance_km	Exponentielle	Distance du déplacement (km)
historique_anomalies	Poisson	Historique d'anomalies passées
ratio_moyenne_dept	Normale	Ratio montant / moyenne du département
alerte_declenchee	Dérivée	Dépassement de seuil automatique
is_fraud	Dérivée	Label fraude (variable cible)

Statistiques clés du jeu de données

5 000 transactions générées avec numpy.random.seed(42)

85 fraudes identifiées, soit un taux de 1,70 % — forte asymétrie de classes

Règle de fraude combinant : montant élevé + week-end + distance + historique d'anomalies

2. Algorithme 1 — Régression Logistique

2.1. Principe et intuition

La Régression Logistique est un modèle statistique simple et hautement interprétable pour la classification binaire. Elle estime la probabilité qu'une note de frais appartienne à la classe fraude via une combinaison linéaire des variables d'entrée, transformée par une fonction sigmoïde :

$$P(\text{fraude} \mid x) = 1 / (1 + e^{-(\beta_0 + \beta_1 x_1 + \cdots + \beta_n x_n)})$$

Elle sert de référence pour mesurer si les relations dans les données sont linéairement séparables.

2.2. Implémentation

```
from sklearn.linear_model import LogisticRegression

# Initialisation et entraînement
lr = LogisticRegression(random_state=42, max_iter=1000)
lr.fit(X_train_scaled, y_train)

# Prédiction et probabilités
y_pred_lr = lr.predict(X_test_scaled)
y_prob_lr = lr.predict_proba(X_test_scaled)[:, 1]
```

2.3. Résultats

98,9 % Accuracy	**88,9 %** Précision
38,1 % Recall (Sensibilité)	**53,3 %** F1-Score
0,951 AUC-ROC	

2.4. Analyse critique

La Régression Logistique affiche une accuracy apparemment élevée (98,9 %) grâce au fort déséquilibre des classes — la grande majorité des transactions étant normales. Toutefois, son recall de seulement 38,1 % révèle une faiblesse structurelle : le modèle manque la majorité des fraudes

réelles. Il détecte les fraudes flagrantes (montants extrêmes) mais échoue sur les patterns croisés et les interactions non-linéaires.

Bilan Régression Logistique

✓Forces : ultra-rapide à entraîner, interprétable via les coefficients β, idéal comme baseline

✗Limites : trop simple pour les interactions complexes, recall insuffisant pour un usage opérationnel

3. Algorithme 2 — Random Forest

3.1. Principe et intuition

Le Random Forest construit un ensemble de multiples arbres de décision indépendants lors de la phase d'entraînement, puis retourne la classe majoritaire par vote. Chaque arbre est entraîné sur un sous-échantillon aléatoire des données (bagging) et sur un sous-ensemble aléatoire des variables. Cet algorithme excelle pour la détection de règles combinées complexes, par exemple :

Si montant > 500 ET weekend = vrai ET distance > 100 km ⇒Fraude

3.2. Implémentation

```
from sklearn.ensemble import RandomForestClassifier

# class_weight='balanced' pour compenser le déséquilibre de classes
rf = RandomForestClassifier(n_estimators=100, random_state=42,
                            class_weight='balanced')
rf.fit(X_train_scaled, y_train)

y_pred_rf = rf.predict(X_test_scaled)
y_prob_rf = rf.predict_proba(X_test_scaled)[:, 1]

# Extraction de l'importance des variables
importances = rf.feature_importances_
```

3.3. Résultats

99,9 % Accuracy	**100 %** Précision (0 faux positif)
95,2 % Recall (Sensibilité)	**97,6 %** F1-Score
0,975 AUC-ROC	

Variables les plus discriminantes selon l'importance des features : montant, ratio_moyenne_departement, historique_anomalies — une information directement exploitable par l'équipe d'audit interne pour prioriser ses contrôles.

3.4. Analyse critique

Le Random Forest domine l'ensemble des algorithmes testés avec un F1-Score de 97,6 % et une précision parfaite de 100 %, signifiant zéro fausse alarme. Le graphique d'importance des variables confirme que le montant, le ratio par rapport à la moyenne du département et l'historique d'anomalies constituent les indicateurs les plus discriminants.

Bilan Random Forest

✓Forces : performances excellentes, robuste aux déséquilibres, explicable via les importances de variables

✗Limites : modèle de type boîte noire, entraînement plus lent, nécessite des données labellisées

4. Algorithme 3 — Isolation Forest

4.1. Principe et intuition

L'Isolation Forest est un algorithme non supervisé qui ne cherche pas à modéliser la normalité, mais à isoler directement les anomalies. Son principe repose sur l'observation que les fraudes sont rares et structurellement différentes, et donc plus faciles à isoler par des partitions aléatoires successives. Un score d'anomalie est attribué : plus un point est isolé rapidement dans l'arbre, plus il est considéré comme anormal.

4.2. Implémentation

```
from sklearn.ensemble import IsolationForest
import numpy as np

# contamination = proportion estimée de fraudes dans les données
iso = IsolationForest(contamination=df['is_fraud'].sum()/len(df),
                      random_state=42)
iso.fit(X_train_scaled)

# -1 = anomalie (fraude), +1 = normal -> remappage
y_pred_iso_raw = iso.predict(X_test_scaled)
y_pred_iso = np.where(y_pred_iso_raw == -1, 1, 0)
y_prob_iso = -iso.score_samples(X_test_scaled)
```

4.3. Résultats

97,9 % Accuracy	**36,8 %** Précision
33,3 % Recall (Sensibilité)	**35,0 %** F1-Score
0,963 AUC-ROC	

4.4. Analyse critique

L'Isolation Forest affiche des résultats modestes en F1-Score (35,0 %) mais réalise cette performance sans jamais avoir eu accès aux labels de fraude. Son AUC de 0,963 est remarquable : en termes de capacité discriminante par score de probabilité, il se rapproche du Random Forest supervisé. Le

faible F1 provient d'un taux élevé de faux positifs — des transactions légitimes mais atypiques signalées à tort.

Bilan Isolation Forest

✓Forces : aucune donnée labellisée requise, excellent AUC, idéal pour une première exploration

✗Limites : trop de faux positifs pour un déploiement direct en production sans validation humaine

5. Comparaison et Évaluation Globale

5.1. Courbes ROC — Comparaison des trois algorithmes

Les courbes ROC permettent de mesurer la capacité discriminante de chaque modèle indépendamment du seuil de décision. L'aire sous la courbe (AUC) constitue le critère synthétique principal.

Synthèse AUC

Les trois algorithmes atteignent des AUC supérieures à 0,95, témoignant d'une excellente capacité discriminante globale.

Random Forest : AUC = 0,975 | Isolation Forest : AUC = 0,963 | Régression Logistique : AUC = 0,951

5.2. Tableau comparatif synthétique

Tab. 2 — Tableau comparatif final des métriques

Algorithme	Accuracy	Précision	Recall	F1-Score	AUC
Régression Logistique	98,9 %	88,9 %	38,1 %	53,3 %	0,951
Random Forest ★	**99,9 %**	**100 %**	**95,2 %**	**97,6 %**	**0,975**
Isolation Forest	97,9 %	36,8 %	33,3 %	35,0 %	0,963

Note : Le Random Forest (★) est mis en évidence comme meilleur algorithme global selon le F1-Score, métrique la plus pertinente en contexte de classes déséquilibrées.

6. Synthèse et Recommandation

Attention — Déséquilibre de classes

L'accuracy élevée de tous les modèles (97–100 %) est en partie due au fort déséquilibre du jeu de données (1,7 % de fraudes seulement).

Le F1-Score est la métrique la plus représentative pour évaluer la qualité réelle de la détection dans ce contexte.

Pour ce cas d'usage de vérification des notes de frais au sein d'une politique RH claire, avec un historique de fraudes confirmées disponible, le Random Forest s'impose comme le meilleur choix :

- F1-Score de 97,6 % — détecte l'écrasante majorité des fraudes réelles
- Précision de 100 % — zéro fausse alarme inutile pour l'auditeur
- Feature Importances — justifie et explique chaque décision de rejet

Note stratégique — Démarrage sans données labellisées

Si l'organisation ne dispose d'aucune donnée historique labellisée, il est recommandé d'utiliser l'Isolation Forest en premier lieu pour identifier les dépenses aberrantes, de les soumettre à une validation humaine, puis d'alimenter un Random Forest avec ces données nouvellement labellisées.

Bibliographie

- Hastie, T., Tibshirani, R., & Friedman, J. (2009). The Elements of Statistical Learning. Springer.
- Géron, A. (2022). Hands-On Machine Learning with Scikit-Learn, Keras & TensorFlow (3e éd.). O'Reilly Media.
- ACFE (2024). Report to the Nations. https://www.acfe.com
- Scikit-learn Documentation. https://scikit-learn.org
- Kaggle — Expense Fraud Detection Dataset. https://www.kaggle.com

Annexe A — Code Source : fraud_analysis.py

Voici le code source complet du fichier fraud_analysis.py, qui génère les données, entraîne les trois modèles et produit toutes les figures du rapport.

```
import pandas as pd, numpy as np
import matplotlib.pyplot as plt, seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier, IsolationForest
from sklearn.metrics import (confusion_matrix, roc_auc_score, roc_curve,
                             accuracy_score, precision_score, recall_score,
f1_score)
import json

# 1. GÉNÉRATION DES DONNÉES (5000 transactions simulées)
np.random.seed(42)
n_samples = 5000
montant = np.random.lognormal(mean=np.log(100), sigma=1.2, size=n_samples)
depense_weekend = np.random.binomial(1, 0.2, size=n_samples)
distance = np.random.exponential(scale=50, size=n_samples)
historique = np.random.poisson(lam=0.5, size=n_samples)
dpt_mean_ratio = np.random.normal(loc=1.0, scale=0.3, size=n_samples)

prob_fraude = (0.3*(montant>500)*depense_weekend +
               0.4*(distance>200)*(historique>2) +
               0.2*(dpt_mean_ratio>3) +
               np.random.uniform(0, 0.05, n_samples))
is_fraud = (prob_fraude > 0.4).astype(int)
depassement_seuil = ((montant>1000)|(dpt_mean_ratio>2.5)).astype(int)

df = pd.DataFrame({
    'montant': montant, 'depense_weekend': depense_weekend,
    'distance_km': distance, 'historique_anomalies': historique,
    'ratio_moyenne_departement': dpt_mean_ratio,
    'alerte_declenchee': depassement_seuil, 'is_fraud': is_fraud})

# 2. PRÉTRAITEMENT
X = df.drop('is_fraud', axis=1); y = df['is_fraud']
X_train,X_test,y_train,y_test = train_test_split(
    X, y, test_size=0.25, random_state=42, stratify=y)
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# 3. RÉGRESSION LOGISTIQUE
lr = LogisticRegression(random_state=42, max_iter=1000)
lr.fit(X_train_scaled, y_train)
y_pred_lr = lr.predict(X_test_scaled)
y_prob_lr = lr.predict_proba(X_test_scaled)[:,1]

# 4. RANDOM FOREST
rf = RandomForestClassifier(n_estimators=100, random_state=42,
                            class_weight='balanced')
rf.fit(X_train_scaled, y_train)
```

```
y_pred_rf = rf.predict(X_test_scaled)
y_prob_rf = rf.predict_proba(X_test_scaled)[:,1]

# 5. ISOLATION FOREST
iso = IsolationForest(contamination=df['is_fraud'].sum()/len(df),
                      random_state=42)
iso.fit(X_train_scaled)
y_pred_iso = np.where(iso.predict(X_test_scaled)==-1, 1, 0)
y_prob_iso = -iso.score_samples(X_test_scaled)

# 6. COURBES ROC
plt.figure(figsize=(8,6))
for y_prob, name in
[(y_prob_lr,'LR'),(y_prob_rf,'RF'),(y_prob_iso,'IsoForest')]:
    fpr,tpr,_ = roc_curve(y_test, y_prob)
    plt.plot(fpr, tpr, label=f'{name}
(AUC={roc_auc_score(y_test,y_prob):.3f})')
plt.plot([0,1],[0,1],'k--'); plt.legend(); plt.savefig('roc_curves.png')
```

Module Intelligence Artificielle — ENCG Settat, CAC 2, S8

Hajbi Douae & Fatima Ezzahraa Mache — M. Abderrahim Larhlimi

Année universitaire 2025–2026